Formalisme de NGeGenBrain_v2

Programmation compositionnelle, expérimentation causale et reconstruction fonctionnelle

Dossier technique NGeGenBrain

Version 2.0 — 27 octobre 2026

Résumé

Ce document formalise l'extension de NGeGenBrain_v1 vers un laboratoire d'étude de modèles neuronaux. Le système conserve le paradigme des programmes déclaratifs, des composants enregistrés, des contrats, des budgets et de la traçabilité, puis ajoute une interface d'adaptation des modèles, l'observation, les interventions, la gestion d'hypothèses, la découverte dans des espaces de règles déclarés, la composition de mécanismes et la reconstruction independante. Le formalisme décrit les objets et les conditions de validité des conclusions ; il ne pose aucune conclusion préalable sur la portée que les expériences pourront atteindre.

Table des matières

1 Objectifs et domaine de validité **2**

2 Lien avec le formalisme compositionnel V1 **2**

3 Modèle étudié et adaptateur **2**

4 Observations et provenance **2**

5 Interventions et effets mesurés **3**

6 Hypothèses et révision **3**

7 Découverte de règles **3**

8 Découverte de circuits et abstraction **4**

9 Langage de mécanismes composables **4**

10 Reconstruction indépendante **4**

11 Séparation des rôles et benchmark aveugle **5**

12 Boucle de recherche active **5**

13 Pipeline canonique **5**

14 Critères de rapport **5**

15 Implémentation actuelle de ce formalisme **6**
15.1 Échelle synthétique et capacités générationnelles 6

16 Reconstruction comportementale d'un modèle causal génératif **6**
16.1 Découverte d'une règle de prochain token 7
16.2 Stratégie scalable de découverte affine 7
16.3 Reconstruction par table finie de logits 7
16.4 Séparation entre enquête et vérité d'audit 7
16.5 Interprétation des statuts 8

17 Principes non négociables **8**

1 Objectifs et domaine de validité

NGeGenBrain_v2 poursuit deux objectifs compatibles mais distincts : (i) organiser des opérations compositionnelles contrôlées, héritées de V1 ; (ii) étudier expérimentalement un modèle neuronal et construire des représentations explicatives ou des exécutables indépendants. Une concordance de sorties constitue une mesure de fidélité comportementale dans un domaine donné ; elle n'est pas, par elle-même, une preuve d'équivalence interne ou mécaniste.

On distingue les objets suivants : observation O, mesure M, hypothèse H, intervention I, prédiction P, résultat R, règle candidate Q, mécanisme K, composition C, modèle reconstruit $\widehat{F}$ et verdict de validation V. Ils sont représentés comme des types distincts dans les registres et ne sont jamais mutuellement convertibles sans une opération explicite et une provenance.

2 Lien avec le formalisme compositionnel V1

V1 représente un programme comme un AST déclaratif et limité, exécuté par un moteur qui résout des composants enregistrés. On note le programme $p \in \mathcal{P}$, son entrée $x \in \mathcal{X}$, son ensemble de capacités Γ et son budget $b \in \mathcal{B}$. La sémantique du runtime est une fonction partielle typée :

$$[\![p]\!]_{\Gamma,b}(x) \in \mathrm{Result}(Y, E),$$

où Y est une valeur typée et E une erreur ou un statut contrôlé. L'existence d'une valeur ne signifie pas que le programme a satisfait toutes les conditions de sortie : le statut, les contrats et les traces appartiennent au résultat.

V2 ne remplace pas cette sémantique. Il ajoute des opérateurs qui produisent des plans d'expérience, des observations, des graphes d'influence, des règles candidates et des compositions exécutables. Les opérations de découverte sont elles-mêmes des composants documentés, testables et traçables.

3 Modèle étudié et adaptateur

Soit un modèle source $F_\theta : \mathcal{X} \to \mathcal{Y}$ muni de paramètres θ. L'accès à F_θ s'effectue par un adaptateur A dont les capacités sont explicites :

$$\begin{aligned}\mathsf{Cap}(A) \subseteq \{&\mathsf{forward}, \mathsf{generate}, \mathsf{inspect}, \mathsf{activations},\\ &\mathsf{attention}, \mathsf{gradients}, \mathsf{intervene}, \mathsf{patch},\\ &\mathsf{ablate}, \mathsf{serialize}, \ldots\}.\end{aligned}$$

Une opération n'est admissible que si sa capacité est annoncée par l'adaptateur et réalisée par celui-ci. Une capacité non disponible n'est ni simulée ni présumée. Deux niveaux d'accès sont particulièrement importants :

- **boîte noire** : l'adaptateur expose des requêtes et leurs résultats ;
- **instrumentable** : l'adaptateur expose en plus certaines structures internes, activations ou interventions.

Le descripteur identifie le modèle, la version, la source, l'empreinte, le nombre de paramètres lorsque connu, l'architecture lorsque observable et la matrice de capacités.

4 Observations et provenance

Une observation est un tuple

$$O = (id, m, h_m, x, y, \phi, c, t),$$

où m est l'identifiant du modèle, h_m son empreinte, x l'entrée, $y = F_\theta(x)$ la sortie observée, ϕ la phase de données, c les captures explicitement disponibles et t l'horodatage. Les observations issues de la génération ou d'un simulateur ne sont pas automatiquement des observations du monde réel : leur origine est inscrite.

Les enregistrements JSONL sont chaînés par empreinte. Pour l'enregistrement r_i,

$$h_i = \mathrm{SHA256}(\mathrm{canon}(r_i \setminus \{h_i\})), \qquad r_i.prev = h_{i-1},$$

avec $h_0 =$ `GENESIS`. Cette chaîne détecte des modifications non coordonnées ; elle ne fournit pas à elle seule une signature d'identité ou une protection contre un acteur pouvant réécrire toute la chaîne.

5 Interventions et effets mesurés

Une intervention est un opérateur I avec une cible explicite, un mode, des paramètres, des contrôles et un contexte d'entrée. Si possible, le protocole observe la paire :

$$y_0 = F_\theta(x), \qquad y_I = I(F_\theta, x),$$

où $I(F_\theta, x)$ désigne l'exécution effective du modèle après l'intervention déclarée (par exemple l'ablation d'une unité ou le remplacement d'une activation). Un effet numérique mesuré peut être :

$$\Delta_2(y_0, y_I) = \sqrt{\sum_j (y_{0,j} - y_{I,j})^2}.$$

Le delta est une mesure d'effet dans les conditions de l'expérience. Il ne suffit pas seul à identifier un mécanisme unique : dépendances indirectes, redondances, interactions, saturation, contexte et effets de compensation restent possibles.

Chaque expérience conserve au minimum l'identifiant de protocole, le hash du modèle et des données, l'hypothèse concernée, l'intervention, les contrôles, les paramètres, les observations, les métriques, l'interprétation, les contre-exemples, l'environnement, l'horodatage et les sources de provenance.

6 Hypothèses et révision

Une hypothèse est un objet versionné :

$$H = (id, \psi, E^+, E^-, \Pi, \mathcal{F}, s, \mathcal{L}),$$

où ψ est une proposition, E^+ l'ensemble des éléments qui la soutiennent, E^- les contre-éléments, Π les effets prédits, $\mathcal{F}$ les conditions de réfutation, s le statut et $\mathcal{L}$ l'historique. Les statuts opérationnels incluent proposée, en test, supportée, contredite, partiellement validée, révisée, abandonnée et consolidée.

Il n'existe pas de transition implicite de « supportée » à « prouvée ». Une transition est un événement journalisé qui conserve l'ancien statut et ses raisons. La confiance est une caractéristique contextualisée ; elle n'est pas une vérité logique.

7 Découverte de règles

La découverte de règles est une recherche dans une classe d'hypothèses $\mathcal{H}$ explicitée. Pour des données de découverte D_{tr}, l'algorithme produit un ensemble de candidats :

$$\mathcal{Q}(D_{tr}, \mathcal{H}) = \{q \in \mathcal{H} : q \text{ satisfait le critère d'ajustement sur } D_{tr}\}.$$

Une classe de recherche restreinte est utile pour rendre une expérience identifiable et reproductible, mais ses résultats ne doivent pas être présentés comme une découverte universelle hors de cette classe. Si aucun candidat exact n'existe, si plusieurs candidats sont équivalents sur les observations ou si les données sont insuffisantes, ces états sont conservés explicitement.

La V2 livrée contient une première induction formalisée de fonctions booléennes affines :

$$f(x_1, \ldots, x_n) = b \oplus \bigoplus_{i \in S} x_i, \qquad S \subseteq \{1, \ldots, n\},\ b \in \{0, 1\}.$$

La recherche énumère les masques S et les deux biais, puis conserve les candidats cohérents avec les lignes de découverte. Le plus simple n'est pas présumé vrai. Les données de validation indépendantes sont nécessaires pour mesurer la généralisation.

8 Découverte de circuits et abstraction

Un graphe d'influence $G = (V, E)$ peut être construit à partir d'interventions. Un arc $u \to v$ enregistre la cible manipulée, la métrique observée, la taille de l'effet, les entrées testées et l'identifiant de l'expérience. L'arc signifie « influence fonctionnelle détectée dans le protocole testé », sauf preuve supplémentaire permettant une interprétation plus forte.

Les niveaux d'abstraction sont indexés par $\ell \in \mathbb{N}$:

$$\text{unités/activations} \to \text{circuits} \to \text{mécanismes} \to \text{sous-systèmes} \to \text{architecture}.$$

L'abstraction est une proposition de regroupement avec une liste de dépendances et de preuves. Une composition de mécanismes doit être retestée comme unité ; les propriétés des composants ne garantissent pas automatiquement celles de leur composition.

9 Langage de mécanismes composables

Un mécanisme est un tuple :

$$K = (id, \ell, X, C, T, Y, D, E, U),$$

où X est l'ensemble des entrées, C les conditions, T une séquence d'opérations déclaratives, Y les sorties, D les dépendances, E les éléments probants et U les informations d'incertitude et de domaine. Une opération doit appartenir à un registre contrôlé ; aucune chaîne de code arbitraire n'est évaluée.

Une composition $K_3 \circ K_2 \circ K_1$ n'est définie que si les sorties requises correspondent aux entrées attendues et si les contrats de types sont satisfaits. Le résultat conserve la provenance et les identifiants de ses sous-mécanismes.

10 Reconstruction indépendante

Une reconstruction $\widehat{F}$ est une implémentation indépendante dérivée des mécanismes ou règles candidates découverts. Elle doit exécuter son propre programme et ne pas interroger secrètement F_θ pendant l'évaluation. Les dépendances de l'artéfact sont énumérées.

On sépare au moins :

1. fidélité de sorties sur les données de découverte ;
2. fidélité sur validation hors-échantillon ;
3. robustesse sur contre-tests et changements de distribution ;
4. fidélité sous interventions homologues ;
5. concordance des structures ou mécanismes lorsque ceux-ci sont observables ;

6. fidélité de trajectoires ou d'états internes, si la comparaison a un sens.

Une métrique comportementale peut être écrite $A_{val} = \frac{1}{|D_{val}|} \sum_{(x,y)\in D_{val}} \mathbf{1}[\widehat{F}(x) = y]$. La valeur de cette métrique ne démontre pas l'équivalence interne, la généralisation à tout domaine ni l'identité des poids.

11 Séparation des rôles et benchmark aveugle

L'environnement de validation sépare : (i) le modèle source ; (ii) le moteur d'analyse ; (iii) les données de découverte ; (iv) les données de validation ; (v) les données de contre-test ; (vi) la vérité de référence détenue par l'auditeur. La vérité cachée ne doit pas être importée par le code de découverte.

Chaque benchmark fixe une graine, publie les versions et hashes pertinents, conserve les exemples sélectionnés, définit la classe d'hypothèses, enregistre les expériences et rend possible leur rejeu. Une vérité de référence n'est consultée qu'après production de la reconstruction, par un auditeur séparé.

12 Boucle de recherche active

Pour plusieurs hypothèses concurrentes $H_1, \ldots, H_k$, le moteur peut proposer une expérience e dont les résultats prédits divergent entre hypothèses. Une fonction d'acquisition peut intégrer l'information attendue, le coût d'expérience, le risque d'intervention et l'incertitude :

$$e^* = \arg\max_e \left(\text{Discrimination}(e; H_1, \ldots, H_k) - \lambda\, \text{Coût}(e)\right).$$

Cette expression définit un objectif de planification, pas une garantie de choix optimal ni de découverte. Le choix, le score et le résultat réel sont enregistrés.

13 Pipeline canonique

```
MODELE SOURCE
   -> identification et matrice des capacites
   -> plan d'observation et empreintes
   -> collecte des observations
   -> hypotheses candidates
   -> plan d'intervention et controles
   -> resultats / contre-exemples
   -> graphes d'influence
   -> regles et mecanismes candidats
   -> composition declarative
   -> reconstruction independante
   -> test holdout and under intervention
   -> audit independant
   -> abstraction superieure et nouveau cycle
```

14 Critères de rapport

Toute conclusion importante doit répondre à :

— qu'est-ce qui a été mesuré exactement ?
— quelle intervention a été effectivement appliquée ?
— quelle prédiction a été enregistrée avant le résultat ?
— quelles hypothèses concurrentes restent compatibles ?

— quels contre-exemples ont été recherchés ?
— quel domaine a été testé et quel domaine ne l'a pas été ?
— le modèle reconstruit est-il réellement indépendant de la source ?
— quel test permettrait de réfuter l'affirmation ?

15 Implémentation actuelle de ce formalisme

Dans la livraison V2, les mécanismes opérationnels comprennent l'adapter PyTorch générique, un adapter facultatif Hugging Face local, un wrapper boîte noire, une capture d'activations par hooks, des interventions scoped, un calcul de delta, des journaux chaînés, la gestion d'hypothèses, la recherche d'une classe booléenne affine, la composition d'opérations whitelistes et le benchmark synthétique de reconstruction fonctionnelle. Le benchmark livré fournit un exemple exécuté de reconstruction fonctionnelle d'une règle XOR sur 4 bits et une validation hors-échantillon.

Il ne constitue pas une reconstruction d'un LLM réel. Le paquet Transformers et les poids d'un LLM n'étaient pas présents dans l'environnement d'essai ; la reconstruction mécaniste d'un LLM complet n'est donc ni déclarée réussie ni déclarée impossible. Elle reste un objectif expérimental à évaluer par les adapters et les protocoles de cette architecture.

15.1 Échelle synthétique et capacités générationnelles

La livraison contient une échelle de huit familles synthétiques, nommées SYNTH-001 à SYNTH-008 : réseau linéaire, parité logique, interactions non linéaires, représentation distribuée, voies concurrentes, MLP hiérarchique, Transformer encodeur miniature et Transformer causal miniature. Le protocole de fumée exerce, pour chaque niveau, une requête boîte noire ; il vérifie sur l'adapter instrumentable la capture d'activations et une intervention locale. Le niveau causal vérifie aussi la génération autoregressive de tokens et l'échantillonnage déterministe à graine fixée.

Cette échelle établit que les interfaces d'observation, d'intervention et de génération sont exécutables sur ces fixtures. Elle ne prétend pas que la découverte mécaniste et la reconstruction ont été validées sur chacune d'elles. Dans cette livraison, seule la fonction de parité 4 bits bénéficie d'une reconstruction symbolique indépendante avec holdout et énumération exhaustive de son domaine fini. Le Transformer causal synthétique utilise des poids aléatoires ; il n'est pas assimilable à un LLM pré-entraîné.

La génération sur l'adapter PyTorch requiert une déclaration explicite `is_causal_lm = True` et une sortie de forme $[B, T, V]$, où B est la taille du batch, T la longueur de séquence et V la taille du vocabulaire. Le moteur choisit le dernier vecteur de logits, applique soit argmax soit un échantillonnage à température positive, puis ajoute le token obtenu. Un modèle qui ne déclare pas cette capacité ne devient pas implicitement génératif. Le wrapper boîte noire peut autoriser les requêtes de génération tout en gardant l'inspection, les activations et les paramètres hors de la matrice de capacités exposée.

16 Reconstruction comportementale d'un modèle causal génératif

Un modèle causal de langage est traité comme une fonction de préfixe. Pour un contexte de tokens $c = (t_1, \ldots, t_k)$, l'adaptateur peut exposer les logits $F_\theta(c) \in \mathbb{R}^V$, où V est la taille du vocabulaire. La génération gloutonne applique récursivement

$$\widehat{t}_{k+1} = \underset{v \in \{0,\ldots,V-1\}}{\arg\max} F_\theta(c)_v, \qquad c \leftarrow \mathrm{suffix}_w(c \,\|\, \widehat{t}_{k+1}),$$

où w est la fenêtre de contexte annoncée et $\|$ désigne la concaténation. Un protocole doit fixer la tokenisation, la fenêtre, le traitement des masques, le choix de génération et les règles de troncature ; ces éléments font partie du contrat expérimental.

16.1 Découverte d'une règle de prochain token

La première reconstruction vise une classe $\mathcal{H}_{\text{aff}}$ finie annoncée avant l'expérience. Dans le cas implémenté, les candidats ont la forme

$$q_{a,b,d}(u, v) = (au + bv + d) \bmod V, \qquad a, b, d \in \{0, \ldots, V - 1\}.$$

L'algorithme ajuste uniquement sur D_{train} et renvoie l'ensemble complet des candidats qui reproduisent toutes les cibles de cet ensemble :

$$\mathcal{Q}_{\text{train}} = \{q \in \mathcal{H}_{\text{aff}} : \forall (c, y) \in D_{\text{train}},\ q(c) = y\}.$$

La règle n'est choisie que si $|\mathcal{Q}_{\text{train}}| = 1$. Si cet ensemble est vide ou contient plusieurs candidats, l'ambiguïté est conservée. Le jeu D_{holdout} ne doit jamais départager les candidats : il sert exclusivement à mesurer la prédiction après l'ajustement.

Le modèle composé exécute sa règle locale et sa boucle autoregressive, sans appel au modèle source après sa construction. On évalue séparément (a) l'exactitude du prochain token sur le holdout, (b) les trajectoires gloutonnes de génération, et (c) la robustesse sur d'autres contextes/interventions. La réussite dans cette classe et ce domaine ne prouve pas que la règle corresponde aux circuits internes utilisés par le réseau source.

16.2 Stratégie scalable de découverte affine

L'énumération exhaustive de tous les triplets (a, b, d) modulo V possède un coût de l'ordre de $O(V^3 n)$ pour n observations et devient impraticable lorsque le vocabulaire est grand. Une seconde classe déclarée, `small_coefficient_modular_affine_v1`, restreint les deux coefficients aux représentants de $\{-1, 0, 1\} \bmod V$. Pour chaque paire (a, b), elle déduit le biais du premier exemple puis vérifie la candidate sur toutes les observations d'entraînement. Le nombre de couples est au plus neuf : le coût est donc $O(9n)$ dans cette classe. Cette recherche n'est pas universelle ; l'absence de candidate ne permet aucune conclusion sur les règles hors de cette classe.

L'API Python propose un sélecteur `auto` qui choisit l'énumération exacte quand $V \leq 32$ et la classe scalable quand $V > 32$. Ce choix dépend uniquement du coût de recherche annoncé, jamais des résultats du holdout. La classe retenue et son domaine sont écrits dans le résultat de l'expérience. Pour tous les modes, les données de validation restent indépendantes et ne sont jamais utilisées pour sélectionner une règle.

16.3 Reconstruction par table finie de logits

Une seconde méthode enregistre pour un ensemble explicite C_{obs} les paires $(c, F_\theta(c))$, puis compose une fonction lookup autonome $\widehat{F}_{\text{tab}}$. Cette méthode fournit une fidélité exacte pour les contextes effectivement interrogés, sous réserve de l'égalité exacte des logits observés et de l'identité des prétraitements. Elle ne fournit pas, par elle-même, une règle générale pour les contextes absents de C_{obs}. L'artefact de reconstruction doit conserver la liste des contextes couverts, les hashes des données et le statut de domaine fini ; une requête hors couverture doit produire une absence explicite, pas une valeur inventée.

Les deux méthodes sont complémentaires : la règle compose une explication concise et testable à l'intérieur d'une classe déclarée ; la table est un témoin comportemental local plus fidèle sur les requêtes stockées, mais peut être volumineuse et ne constitue pas une induction de mécanisme. Dans les deux cas, le modèle composé ne copie pas les poids du modèle source et ne consulte pas l'oracle pendant l'inférence.

16.4 Séparation entre enquête et vérité d'audit

Le constructeur du banc peut connaître la règle utilisée pour entraîner le modèle synthétique. Cette vérité est placée dans un répertoire d'audit séparé et n'est pas incluse dans les entrées du

moteur de découverte. Le moteur reçoit exclusivement un adapter de requête, les contextes pré-enregistrés et les logits/sorties observés. Le rapport public décrit la méthode et les scores mesurés sans révéler la vérité cachée avant que le processus d'évaluation ne le permette.

16.5 Interprétation des statuts

Les statuts sont contextualisés :

— `VALIDATED_GREEDY_BEHAVIOR_ON_HELDOUT_AND_COMPOSED_TRAJECTORIES` signifie que la règle unique trouvée dans la classe déclarée a réussi les tests de prochain token et de génération gloutonne réservés par ce protocole ;

— `VALIDATED_ON_QUERIED_FINITE_CONTEXT_DOMAIN_AND_COMPOSED_TRAJECTORIES` signifie que les logits/trajectoires ont été reproduits dans le domaine fini explicitement interrogé ;

— aucun de ces statuts n'est une preuve d'équivalence interne, de récupération des poids, de reconstruction complète d'un LLM de taille réelle, ni de validité hors du domaine spécifié.

Les résultats doivent enregistrer les hashes du modèle source, du jeu de contextes, des prompts, des artefacts composés, de la configuration et de l'environnement, ainsi que les contextes non couverts et les contre-tests. Une reconstruction ne peut être dite indépendante que si l'inférence du modèle composé n'appelle pas le modèle source et si cette propriété est vérifiée/testée.

17 Principes non négociables

1. Toute capacité est adossée à un adapter et une déclaration vérifiable.
2. Toute intervention est enregistrée et ne peut être simulée silencieusement.
3. Les hypothèses, règles candidates, faits et résultats sont des catégories distinctes.
4. Les données de test ne participent pas à l'ajustement de la règle évaluée.
5. La vérité du benchmark est isolée des processus de découverte.
6. Les reconstructions indiquent leur dépendance au modèle source et leur domaine.
7. La traçabilité est vérifiable et les erreurs restent conservées.
8. Les limitations observées sont consignées sans être codées comme plafond théorique du système.